1821 Universidad de Buenos Aires

.UBAeconómicas | posgrado

ENAP Escuela de Negocios y Administración Pública



Del PDF a la respuesta

RAG mínimo con Ollama, en ~200 líneas de Python

Mg. Ezequiel Nuske · Ex Chief Data Officer

Laboratorio interdisciplinario de IA en organizaciones · 21 de Mayo de 2026

Agenda

1. El problema: LLMs que no conocen **tus** documentos
2. La hipótesis: **recuperar antes de generar**
3. Anatomía de un RAG mínimo en 5 pasos
4. Cada componente en código
5. Trade-offs, límites y próximos pasos

El problema en una slide

Qué falla

- LLMs entrenados con datos públicos
- Sin acceso a tus PDFs, mails, wikis
- *"No lo sé"* — o peor: **alucinaciones**

Qué intentamos antes

- Fine-tuning: caro, lento, opaco
- Prompt con todo el PDF: no entra
- Buscador clásico: sin comprensión semántica

La pregunta: ¿cómo le damos al LLM acceso a información privada, *sin* reentrenarlo y *sin* mandarlo todo en cada prompt?

> *Necesitamos un puente entre **tus documentos** y **la generación**.*

Un caso concreto

- Un investigador con **50 PDFs** de tesis y papers.
- Pregunta típica: *"¿Qué dicen los autores sobre el sesgo por dispositivo?"*
- Leerlos todos: **días**.
- Pegarlos en ChatGPT: **no entran** en el contexto.
- Buscar por palabra clave: pierde sinónimos y contexto.

Lo que queremos: una respuesta en segundos, con **citas** al PDF y al fragmento exacto.

La hipótesis

En lugar de reentrenar al modelo,
recuperamos los fragmentos relevantes
y **aumentamos** el prompt con ellos.

Eso es **RAG**: *Retrieval-Augmented Generation*.

¿Qué es RAG, en una idea?

Fase 1 · Indexing *(offline)*

- Extraer texto de los documentos
- Partirlo en fragmentos (*chunks*)
- Convertir cada fragmento en un **vector** (embedding)
- Guardar `vector → texto`

Fase 2 · Querying *(online)*

- La pregunta también se convierte en vector
- Buscamos los **chunks más parecidos**
- Los pegamos en el prompt como contexto
- El LLM responde **anclado** a ese material

> *El modelo no "sabe" más. Sabe **buscar mejor**.*

¿Por qué hacerlo local con Ollama?

Privacidad

- Los PDFs **no salen** de tu máquina
- Sin API keys, sin telemetría
- Apto para datos sensibles *(legales, médicos, internos)*

Costo

- $0 por consulta
- Sin límites por tokens

Control

- Elegís el LLM y el embedder
- Reproducible y versionable
- Funciona **offline**

Aprendizaje

- Cada pieza es visible y editable
- Ideal como base didáctica antes de subir a producción

Anatomía de un RAG mínimo



Cinco pasos, cada uno con una responsabilidad clara:

1. **Extraer** texto del PDF
2. **Trocear** en *chunks* manejables
3. **Embeddear** cada chunk
4. **Recuperar** los más parecidos a la pregunta
5. **Generar** la respuesta con el contexto recuperado

Stack del proyecto

Pieza	Tecnología	Por qué
Extracción de PDF	`pypdf`	Texto plano, sin dependencias pesadas
Vectores	`numpy`	Suficiente para `cosine` con `N` chico
LLM + Embeddings	**Ollama** *(HTTP)*	Local, gratis, intercambiable
Glue	`requests` + 188 líneas	Sin frameworks, todo legible

Modelos *(por defecto)*: `nomic-embed-text` para embeddings, `deepseek-r1` o `llama3.1` para generación.

> *Sin vector DB, sin LangChain, sin LlamaIndex.* ***A propósito.***

Paso 1 · Extracción

Sacamos texto del PDF y limpiamos los espacios sueltos. `pypdf` es suficiente para PDFs con capa de texto; los escaneados necesitan OCR aparte.

```
def extract_text_from_pdf(path: str) → str:
    reader = PdfReader(path)
    pages = []
    for i, page in enumerate(reader.pages):
        txt = page.extract_text() or ""
        pages.append(txt)
    text = "\n".join(pages)
    text = re.sub(r"\s+\n", "\n", text)
    text = re.sub(r"\n\s+", "\n", text)
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()
```

> *Regla: si la extracción es ruidosa, **todo lo demás sufre**.*

Paso 2 · Chunking con overlap

Cortamos en bloques de ~1200 caracteres, solapando 200. El *overlap* evita perder ideas que cruzan el corte; el tamaño balancea contexto vs. ruido.

```
CHUNK_SIZE_CHARS = 1200
CHUNK_OVERLAP_CHARS = 200

def chunk_text(text: str, chunk_size: int, overlap: int) → list[str]:
    if not text:
        return []
    chunks = []
    start, n = 0, len(text)
    while start < n:
        end = min(start + chunk_size, n)
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        if end == n:
            break
        start = max(0, end - overlap)
    return chunks
```

> *Chunk demasiado chico → fragmenta ideas. Demasiado grande → mete ruido en el contexto.*

Paso 3 · Embeddings con Ollama

Convertimos cada chunk en un vector. Llamamos al endpoint `/api/embed` con `nomic-embed-text` y reintentamos con *exponential backoff* — Ollama puede tardar mientras carga el modelo.

```
def ollama_embed(text: str, retries: int = 5) → np.ndarray:
    url = f"{OLLAMA_BASE_URL}/api/embed"
    payload = {"model": EMBED_MODEL, "input": text}
    for attempt in range(retries):
        try:
            r = requests.post(url, json=payload, timeout=120)
            r.raise_for_status()
            emb = r.json()["embeddings"][0]
            return np.array(emb, dtype=np.float32)
        except (HTTPError, ConnectionError):
            wait = 2 ** attempt        # 1, 2, 4, 8, 16 s
            if attempt < retries - 1:
                time.sleep(wait)
                continue
            raise
```

> *El embedder define qué quiere decir "parecido". Cambiarlo cambia **todo el sistema**.*

Paso 4 · Búsqueda semántica

Normalizamos los vectores una vez y la similitud coseno colapsa a un *dot product* — un único `mat @ q` ordena todo el corpus.

```
def build_index(pdf_paths):
    vectors, docs = [], []
    for path in pdf_paths:
        for j, ch in enumerate(chunk_text(
                extract_text_from_pdf(path), 1200, 200)):
            vectors.append(ollama_embed(ch))
            docs.append({"source": path, "chunk_id": j, "text": ch})
    mat = np.vstack(vectors)
    mat = mat / np.linalg.norm(mat, axis=1, keepdims=True)
    return {"docs": docs, "mat": mat}

def retrieve(index, query, top_k):
    q = ollama_embed(query); q = q / np.linalg.norm(q)
    scores = index["mat"] @ q
    top = np.argsort(scores)[::-1][:top_k]
    return [index["docs"][int(i)] | {"score": float(scores[i])} for i in top]
```

> *Con `N` chico (cientos a miles de chunks), `numpy` es **más rápido** que montar Postgres + pgvector.*

Paso 5 · Prompt y generación

Instrucciones explícitas al LLM: respondé *solo* con lo recuperado y, si no alcanza, decilo. Las **citas a fuente + chunk** convierten al RAG en algo *auditable*.

```
def build_rag_prompt(question, contexts):
    ctx = "\n\n".join(
        f"[Fuente: {c['source']} | chunk {c['chunk_id']} | score {c['score']:.3f}]\n{c['text']}"
        for c in contexts
    )
    return f"""Sos un asistente. Respondé usando SOLO la información del CONTEXTO.
Si el contexto no alcanza, decí "No lo sé con este material".

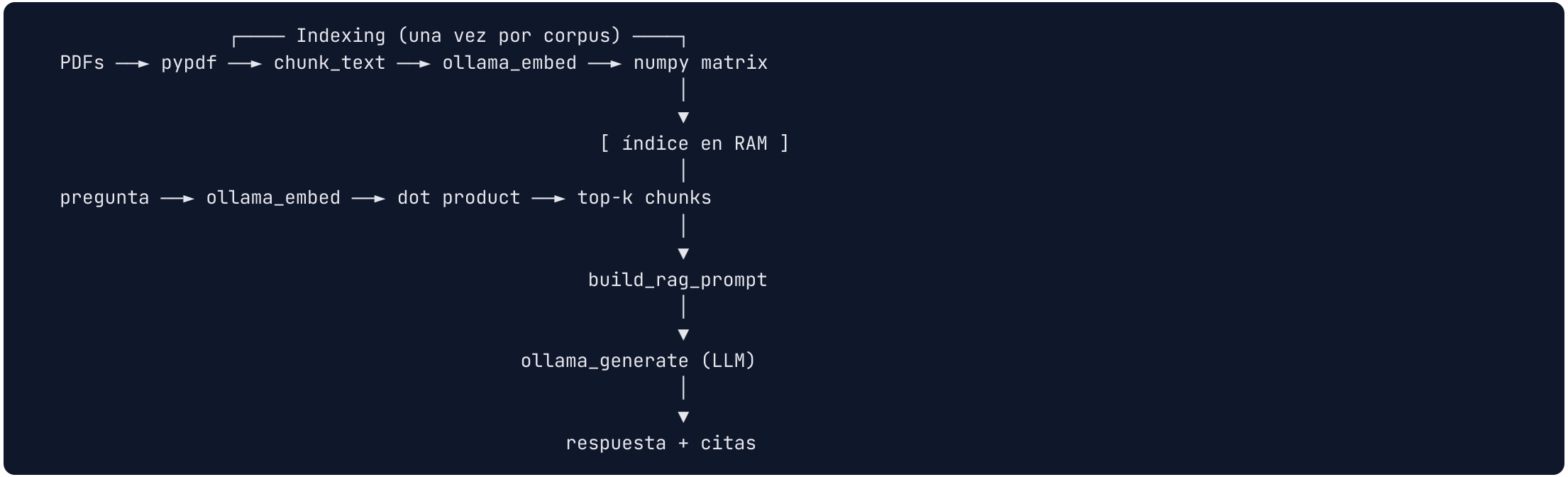
CONTEXTO:
{ctx}

PREGUNTA:
{question}

RESPUESTA (en español, concisa, con referencias a fuente y chunk):
"""
```

> *El LLM ahora opera **anclado a tu material**: sin contexto, no inventa.*

Arquitectura del flujo



Camino caliente *(segundos)*: embed pregunta → dot → top-k → LLM.

Camino frío *(una vez)*: extract → chunk → embed → matriz normalizada.

Trade-offs · chunking

Decisión	Si va **bajo**	Si va **alto**
Tamaño del chunk *(chars)*	Pierde contexto, fragmenta ideas	Mete ruido, gasta tokens del LLM
Overlap	Corta razonamientos en los bordes	Duplica información, infla el índice
`top_k`	El LLM se queda corto	Contexto contradictorio, latencia + costo

Default usado: `chunk=1200`, `overlap=200`, `top_k=5`.

> *No hay valores universales. Hay que **medir** contra preguntas reales del dominio.*

Trade-offs · embeddings

Dimensión del vector

- 384 → barato, búsquedas rápidas
- 768–1024 → mejor calidad semántica
- 3072+ → caro, marginal en dominios chicos

Dominio

- Modelos generales: español OK
- Dominios técnicos *(legal, médico)*: conviene fine-tune o modelos específicos

Velocidad vs. calidad

- `nomic-embed-text` *(768d)*: equilibrio razonable, gratis local
- Modelos comerciales: mejor calidad, pero envían tu texto afuera

Implicancia clave

- Cambiar de embedder **invalida el índice**: hay que rebuildear todo
- Tratar el embedder como **dependencia versionada**

Por qué cosine similarity

- Normalizamos cada vector una vez: `v / ‖v‖`
- `cos(a, b) = a · b` cuando `‖a‖ = ‖b‖ = 1`
- Un único producto matricial `mat @ q` puntúa **todo el corpus**
- Argsort + slice → `top_k` en una línea

Ventaja práctica: para `N` chico (cientos a miles de chunks) supera a Postgres + pgvector en simpleza *y* en latencia.

> *Cuando `N` crece a millones, ese mismo `mat @ q` se vuelve inviable y entran FAISS / Qdrant / pgvector.*

Cuándo NO alcanza este RAG mínimo

Escala

- Millones de chunks → necesitás un *vector store* con índices (FAISS, Qdrant, pgvector)
- Multi-usuario concurrente → API + cache + cola
- Documentos que cambian → re-indexado incremental

Calidad de retrieval

- Sinónimos o jerga: agregar **BM25 híbrido**
- Resultados ambiguos: **reranking** *(cross-encoder)*
- Preguntas multi-hop: *query rewriting*

Operación

- No sabés si el RAG mejoró nada sin **evaluación** *(Ragas, golden set)*
- Sin **observabilidad** *(logs de hits, scores, latencias)* es imposible debuggear
- Costo y privacidad: si vas a la nube, contrato + DPA

Seguridad

- Inyección de prompt vía contenido del PDF
- Filtrado por permisos del usuario sobre los documentos

Próximos pasos naturales

Retrieval

- BM25 + denso *(hybrid search)*
- Reranking con cross-encoder
- *Query rewriting* con el propio LLM
- Filtros por metadata *(fecha, autor, sección)*

Indexing

- Persistir índice en disco *(np.save / FAISS)*
- Re-indexado incremental
- Chunking estructural *(por título / sección)*

Evaluación

- Golden set de preguntas con respuesta esperada
- Métricas: *context recall*, *faithfulness*, *answer relevance*
- A/B de configuraciones *(chunk, embedder, top_k)*

Producto

- API + UI mínima
- Citas clickeables al PDF original
- Caching de embeddings y de respuestas frecuentes

1821 Universidad de Buenos Aires

.UBAeconómicas | posgrado

ENAP Escuela de Negocios y Administración Pública



El número que importa

~200

líneas de Python, $0 de infra, 0 vendor lock-in.

Un RAG honesto, auditable y modificable.

Suficiente para prototipar, enseñar y validar la idea antes de escalar.

Lecciones aprendidas

1. **Empezar mínimo** vale la pena: entender cada pieza antes de meter un framework.
2. **El embedder define el sistema**: cambiarlo = rebuild + revalidación.
3. **Las citas no son cosmética**: hacen al RAG *auditable* y diagnosticable.
4. `numpy` **alcanza** hasta los miles de chunks. Solo después aparece el *vector store*.
5. **Privacidad por diseño** con Ollama local: ideal para dominios sensibles.
6. **Sin evaluación, no sabés si mejoraste nada.** El golden set es el primer paso *después* del MVP.

1821 Universidad de Buenos Aires

.UBAeconómicas | posgrado

ENAP Escuela de Negocios y Administración Pública



Para la discusión

¿Qué corpus privado tenés hoy donde un asistente con **citas a la fuente** cambiaría cómo trabajan las personas?

¿Qué decisión del pipeline *(chunk, embedder, top-k, prompt)* haría más diferencia en **tu** dominio?

1821 Universidad de Buenos Aires

.UBAeconómicas | posgrado

ENAP Escuela de Negocios y Administración Pública



¡Gracias!

Ezequiel Nuske

Email
eonuske@mail.austral.edu.ar

LinkedIn
https://www.linkedin.com/in/ezequiel-nuske-15137862/

Preguntas y respuestas